

Izveštaj – Projekat2 – Projektovanje i analiza algoritama

Zadatak: Implementirati po priloženoj tabeli, gde prva kolona predstavlja cifru jedinicu broja indeksa, tri algoritma iz odgovarajuće vrste za rešavanje navedenog problema. Problem zadat u nastavku rešiti primenom tri različita algoritma za sortiranje i uporediti njihove performanse po vremenu izvršenja i utrošku memorije. Napisati i predati kratak izveštaj poređenja performansi algoritama za različite ulazne nizove. Za sortiranje koristiti celobrojne slučajne vrednosti iz opsega od 0 do 10000. Sortirati slučajno generisane nizove celih brojeva od 100, 1000, 10k, 100k, 1M i 10M elemenata.

Algoritmi za sortiranje iz tabele: Selection, Heap i Counting.

Problem1: Fabrika čokolade je predstavljena celobrojn timer nizom od N elemenata, gde i-ti element predstavlja broj čokolada u i-tom paketu. Grupa od M studenata je došla u posetu fabrici i po izlasku svaki student treba da dobije tačno jedan paket. Raspodeliti pakete studentima tako da razlika između maksimalnog i minimalnog broja čokolada u paketima datim studentima bude minimalna. Prikazati dobijenu razliku. Uzeti da je M 30% vrednosti N.

1. Vreme izvršenja

Slučajno generisan niz celih brojeva od 100 elemenata:

```
Unesite broj paketa: 100
Broj paketa je:100
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 2.59e-05 s
Minimalna razlika je: 1882
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.0001376 s
Minimalna razlika je: 1929
-----HeapSort-----
Vreme izvršenja HeapSort-a je 2.33e-05 s
Minimalna razlika je: 2071
```

Slučajno generisan niz celih brojeva od 1000 elemenata:

```
Unesite broj paketa: 1000
Broj paketa je:1000
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 0.0022994 s
Minimalna razlika je: 2591
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.0001194 s
Minimalna razlika je: 2741
-----HeapSort-----
Vreme izvršenja HeapSort-a je 0.0003493 s
Minimalna razlika je: 2734
```

Slučajno generisan niz celih brojeva od 10k elemenata:

```
Unesite broj paketa: 10000
Broj paketa je:10000
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 0.21407 s
Minimalna razlika je: 2923
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.0003209 s
Minimalna razlika je: 2953
-----HeapSort-----
Vreme izvršenja HeapSort-a je 0.0069095 s
Minimalna razlika je: 2970
```

Slučajno generisan niz celih brojeva od 100k elemenata:

```
Unesite broj paketa: 100000
Broj paketa je:100000
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 22.8297 s
Minimalna razlika je: 2958
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.0015857 s
Minimalna razlika je: 2979
-----HeapSort-----
Vreme izvršenja HeapSort-a je 0.0479412 s
Minimalna razlika je: 2968
```

*u ovom delu je poziv SelectionSorta bio zakomentaran kako bi se prikazalo vreme za Heap i Counting

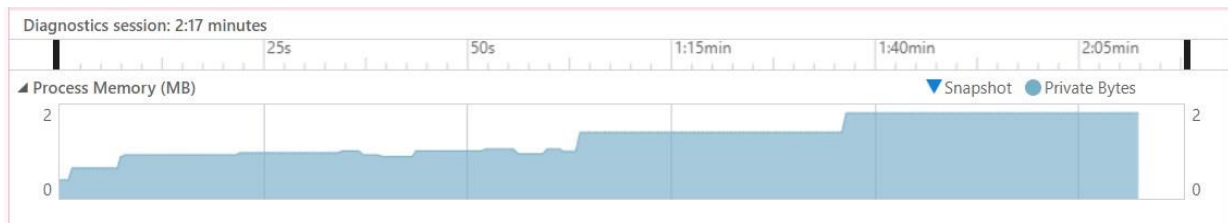
Slučajno generisan niz celih brojeva od 1M elemenata:

```
Unesite broj paketa: 1000000
Broj paketa je:1000000
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 5e-07 s
Minimalna razlika je: -9986
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.0307999 s
Minimalna razlika je: 2994
-----HeapSort-----
Vreme izvršenja HeapSort-a je 1.0926 s
Minimalna razlika je: 2992
```

Slučajno generisan niz celih brojeva od 10M elemenata:

```
Unesite broj paketa: 10000000
Broj paketa je:10000000
-----SelectionSort-----
Vreme izvršenja SelectionSort-a je 4e-07 s
Minimalna razlika je: -9994
-----CountingSort-----
Vreme izvršenja CountingSort-a je 0.26478 s
Minimalna razlika je: 2997
-----HeapSort-----
Vreme izvršenja HeapSort-a je 15.9208 s
Minimalna razlika je: 2997
```

2. Utrošak memorije



Vidimo da se memorijski peak povećava tokom rada programa jer se u samom kodu prvo izvršava selection sort, zatim heap sort i na kraju counting sort. Selection sort i heap sort nemaju nikakve dodatne memorijske alokacije, dok counting sort upotrebljava dodatni pomoćni niz, zbog koga u tom delu izvršenja vidimo porast u memorijskoj potrošnji. Promene svakako nisu velike zbog ispravnog oslobađanja memorije pri svakom pokretanju koda. U kodu je izvršena dealokacija memorije za svaku promenljivu u dinamičkoj zoni memorije, zbog čega na grafu vidimo da nema porast u utrošku memorije.

Rešenje problema:

Dati problem zahteva traženje podniza u nizu. Inicijalno postavljamo da je minimalna razlika jednaka razlici poslednjeg elementa prvog podniza M i prvog elementa tog istog podniza. Dolazimo do prave minimalne razlike iteracijom kroz celokupan niz, od prvog do $N-M$ -tog indeksa, ponavljanjem istog procesa: oduzimanje prve vrednosti podniza od poslednje vrednosti podniza. Ovim smo obezbedili da prođemo kroz svaki podniz dužine M koji se nalazi unutar niza Fabrika. Svakom iteracijom kreiramo novi početak podniza. Niz Fabrika je sortiran niz, dakle znamo da se unutar svakog podniza na prvoj poziciji nalazi najmanji element, a na poslednjoj najveći element za taj podniz. Sortiranje nam takođe obezbeđuje rastući poredak elemenata niza, koji nam omogućava pravilno računanje minimalne razlike između minimalnog i maksimalnog elementa podniza.

Za svaki sorting algoritam pokrenula sam novu funkciju za generisanje nasumičnih vrednosti kako bi svaki algoritam mogao da izvrši sortiranje i pokazao svoje performanse.

Za merenje vremena upotrebljena je biblioteka `<chrono>`. Za generisanje nasumičnih vrednosti upotrebljeni su `random_device`, `mt19937` i `uniform_int_distribution` kako bismo osigurali dobru raspodelu vrednosti od 0 do 10 000.

Zaključak:

Selection sort

Selection sort algoritam funkcioniše tako što vrši pretragu unutar celog niza za najmanji element i postavlja ga na prvo mesto. U narednoj iteraciji, u preostalom delu niza traži drugi najmanji broj i postavlja ga na drugo mesto. Ovaj proces se ponavlja dok ne dođe do kraja niza.

Selection sort ima složenost $O(n^2)$. Po rezultatima možemo videti da je selection sort manje efikasan u odnosu na druga dva algoritma i da je bilo neophodno mnogo više vremena da se izvrši sortiranje. Za nizove velikih dužina, poput niza dužine 1M i 10M selection sort ima vreme izvršenja od nekoliko sati što je ekstremno vremenski i memorijski neoptimalno. Sortiranje se izvršava ovoliko dugo zbog kvadratne složenosti ovog algoritma. Kako raste veličina niza n , tako će rasti i vreme izvršenja ovog algoritma.

Možemo da zaključimo da ovakav algoritam pokazuje bolje performanse za nizove manjih dužina.

Heap sort

Heap sort algoritam koristi gomilu (heap). Kako bi ovaj algoritam funkcionisao, neophodno je kreirati binarno stablo – gomilu (svaki roditeljski čvor je veći ili jednak svojoj deci). Nakon formiranja heap-a, znamo da se u korenu nalazi najveći element (max heap). Koren heap-a (maksimalni element) menjamo sa poslednjim elementom niza. Nakon ovoga neophodno je da se veličina heap-a smanji i da se ostali čvorovi prilagode za novi max heap. Ovaj proces se ponavlja sve dok svi elementi ne budu sortirani.

Heap sort ima složenost $O(n \log n)$. Heap sort je veoma efikasan algoritam za sortiranje, međutim kada sortiramo nizove manjih dužina, zbog vremena koje je utrošeno na formiranje gomile, drugi algoritmi će biti efikasniji. Sa povećanjem broja elemenata, heap omogućava efikasno izvršenje sortiranja. U pogledu vremenskog izvršenja sa veoma velikim nizovima, heap sort će ostati stabilan, ali će imati lošije performanse u pogledu pristupa memoriji zbog načina na koj pristupa memoriji. Gomila se formira tako što od i -tog čvora, njegove potomke smestimo na pozicije $2i$ i $2i+1$, što može izazvati često skakanje po memoriji.

Counting sort

Counting sort je algoritam koji sortira vrednosti unutar poznatog opsega. Algoritam prolazi kroz ulazni niz i unutar pomoćnog niza pamti broj pojavljivanja svakog elementa. Pomoćni niz se kreira tako što se vrši kumulativno sumiranje svih prethodnih elemenata u nizu. Ovakvom metodom se određuje željena pozicija svakog elementa. Na kraju se ide unazad kroz inicijalni niz i brojevi se iz pomoćnog niza smestaju na svoje mesto. Kada smestimo neki element na svoje mesto, neophodno je da smanjimo njegov broj ponavljanja u pomoćnom nizu. Ovako omogućavamo da se svaki element sortira i smesti na korektno mesto.

Counting sort ima složenost $O(n+k)$, gde je n broj elemenata u nizu, a k najveća vrednost u opsegu. Sa nizom od 100 elemenata counting sort je duže vršio sortiranje jer duže traje popunjavanje pomoćnog niza za male vrednosti (drugi algoritmi imaju prednost sa veoma malim vrednostima). Counting sort zadržava svoju linearnu složenost i sa jako velikim veličinama niza n i ostaje vremenski efikasan. Memorijski counting sort zahteva dodatni niz za izlaz, što smanjuje njegovu optimizaciju memorije, ali je i dalje efikasniji u odnosu na heap sort. Ovaj algoritam zavisi u velikoj meri od broja elemenata k , u ovom slučaju nama je k 10 000, ali sa povećanjem vrednosti k , smanjila bi se efikasnost ovog algoritma.

Ukoliko postoji prostor u memoriji u kom se može pamtiti pomoćni niz, counting sort je veoma efikasan i brz za velike nizove.